Tennis Scoring Program

October 11, 2024

Contents

Tennis Scoring Program

Com S 2270 (Fall 2024) Programming Assignment 2 (200 points)

- Due Date: Monday, October 14th, 11:59 pm (midnight)
- 5% bonus for submitting 1 day early (by 11:59 pm October 13th)
- 10% penalty for submitting late (by 11:59 pm October 16th)
- No submissions accepted after October 16th, 11:59 pm

Clarifications

This document has been updated with a few clarifications which were made on Piazza.

- You should not implement a method called `getPlayerAPoints` or `getPlayerBPoints` which were mentioned in an earlier version of this document. Instead you should implement `getPlayerAScore` and `getPlayerBScore`, which are described in the javadoc. You may choose to implement the former methods as private methods if you think that is useful.
- Scores during tiebreaks (and zeros) which have only one digit must be printed with a space in front (as returned by `getPlayerAScore` and `getPlayerBScore`. However when printed in the returned string from `getCallOut`, they should be without any extra space padding. So you should for example return `2-1 PlayerName` from `getCallout`.

- The unit tests have been modified – they no longer do a string comparison of the scoreboard to the expected scoreboard. Instead they directly compare the numbers that are supposed to be in the scoreboard, as ints (with the exception of the scores which are strings, and in that case a string comparision is used). The only thing you should notice is that the tests tell you precisely what is different between the expected values and the values produced by your solution.

Preamble

This assignment is to be done on your own. See the Academic Integrity policy in the syllabus, for details. You will not be given credit unless you have completed the Academic Dishonesty policy questionnaire on the Assignments page on Canvas. Please do this right away.

If you need help, try office hours, the schedule is on Canvas. Lots of help is also available through the Piazza discussions. Please start the assignment as soon as possible to get your questions answered right away. It is physically impossible for the staff to provide individual help to everyone the afternoon that the assignment is due!

This is a "regular" assignment so we are going to read your code. Your score will be based partly on the autograder's functional tests and partly on the grader's assessment of the quality of your code. See the "More about grading" section.

Overview

The purpose of this assignment is to give you lots of practice working with conditional logic and managing the internal state of a class. You'll create one class, called Tennis, that is a simplified model of the scoring system for the game of Tennis. Although there is a strong resemblance to the game of Tennis, it's probably best if you forget everything you know about Tennis from the real world as you read these instructions. Your job is to implement the rules of the game that is specified in this document even if that conflicts with your understanding of any other Tennis rules.

The following rules for scoring a tennis match are adapted from the USTA Scoring Rules at www.usta.com.

`https://www.usta.com/en/home/improve/tips-and-instruction/national/tennis-scoring-rules.html`

You may read that page for additional clarity, but when there is a substantive difference, prefer this document for the purposes of doing this programming assignment.

For our purposes, Tennis is a two-player sport. The aim of tennis is to win enough points to win a game, enough games to win a set, and enough sets to win a match.

Scoring a Match – the point system

Tennis has two ways of counting things. Normal counting by using the numbers 0, 1, 2, ... is used for games, sets and points in tiebreaks, but most of the time the points in a game are counted using the following nonconsecutive numbers:

0 when no points have been won, 15 when one point has been won, 30 when two points have been won, and 40 when three points have been won.

When the score is 0, this is read aloud as "Love". Generally when two players have won the same number of points, we say "(the score)-All", with the exception that when players are tied at 40, we say "Deuce". When the score is "Deuce", a player must win two more points than the opponent in order to win. When the player has won one point more than the opponent, we say that player has an "Advantage" (and this is called out as "Advantage playerName"). If the player wins an additional point, they win the game, otherwise, the score returns to "Deuce".

Who Wins?

A player can win a game by winning a point after getting a score of 40 provided the opponent has not yet obtained a score of 40. If both players get a score of 40, then the rules discussed for "Deuce" above are observed. Other than "Deuce", if the players have the same number of points, or the same number of games, we use the word "All" to call out the score. For example the score could be "Thirty-All", or "Fifteen-All".

Types of Tennis Sets

A player needs to win enough games to win a set, and must have a minimum margin of victory in terms of games. There are two systems for winning a set, and during a tennis match we only use one of them throughout the match.

Advantage Set

To win a set under the Advantage system, a player must win at least six games, and lead by a margin of at least two games in order to win the set. This can lead to long sets when players are tied after every other game.

Tiebreak Set

When playing under the Tiebreak system, a player can win a set with six or seven games when leading by a margin of at least two. If the game score gets to 6-all, then a tiebreak game is played. During a tiebreak game, points are counted normally (using 0, 1, 2, 3, ...), and a player must win the game by at least 7 points with a point-margin of at least two in order to win the last game of the set and with it, the set.

Grand Slam Matches

Grand Slam Matches were inconsistent in the past in terms of how their final set is completed when the score is six games all. It used to be the case that Advantage Sets were played, and as a result, the longest match in tennis history took three days to complete!

Recently Grand Slam matches have been standardized to a tie-break format where a player needs to win at least 10 points by a margin of at least two, in order to win a Grand Slam match. This latter rule is what we will model in our program.

Implementation Details

Your goal is to implement a class called `Tennis`. The players are called `playerA` and `playerB`. The match will begin with playerA serving, and after that they toggle the role of serving at the end of every game (with an exception for tiebreaking which is explained later). This class will have methods that are called to indicate the following:

```
public void winPoint(boolean playerAWins);
```

Calling winPoint with a true argument means that playerA wins a point, and that must trigger a scoreboard update, and a new callout. Calling it with a false argument means playerB wins the point. This function will be called repeatedly to reflect the sequence in which points are won by the players.

When this method is called, it should determine whether winning the point implied that they also won the game. If they did win the game as well, it should also determine whether that means they also won the set. If they won the set, as well, it should also determine whether that means they won the match.

We also want to be able to model the match at a higher level in terms of games won by making repeated calls to the following method (as if someone were telling you who won each game, but not telling you who won each point):

```
public void winGame(boolean playerAWins);
```

Calling it with a true argument means playerA won a game, while a false argument means playerB won a game. This method only tracks the numbers of games won, the number of sets won and the conclusion of the match. If a game is won, the method should determine whether that means a set was won, and if so, whether the match was won.

Finally the simplest case of modelling is by making repeated calls to

```
public void winSet(boolean playerAWins)
```

Here too, we use the convention is that a true argument means playerA won a set, and a false argument means playerB won a set. This just counts the sets, and determines who wins the match by comparing the number of sets won. In this case you determine the sequence of calls to make by pretending someone is only telling you who won each set, but not telling you who won each game or point.

Scoring

This homework will separately track how scores are announced orally (which we will refer to as "callouts"), and how scores are updated on a scoreboard (which we call "scoreboard").

For oral scores, we assume the callouts are done in English, and use the following words:

- 0 points = Love
- 1 point = Fifteen (the word, not the number)
- 2 points = Thirty

- 3 points = Forty
- Tied score = All
- 40-40 = Deuce
- If playerX wins deuce point = Advantage *<playerXName>*

So the following are examples of valid callouts:

```
Fifteen-Forty
Deuce
Thirty-All
Forty-Love
Advantage <playerXName>
```

while the following are not:

```
Forty-Forty
Forty-All
```

The exception is when a tiebreaker is being played, in which case we use normal counting numbers to announce the scores, like 2-1, 2-2 and so on. Valid callouts list the higher score first, a dash, the lower score, a space and then the name of the leading player. If the scores are equal, then we simply callout the score followed by "-All". Examples of valid callouts during tiebreaks are:

```
2-1 Alice
2-All
3-2 Betty
```

We will also assume that at the end of a Game and before the next one, the callout announces the winner of the previous Game instead of the starting score of the next game, so the following are valid callouts:

```
Game: <playerXName>
Game and Set: <playerXName>
Game, Set and Match: <playerXName>
```

where `<playerXName>` is the name of the player that won that part of the match. For this reason, the following are not valid callouts (for the purposes of this assignment):

```
Love-Love
Love-All
```

We note in passing that all or most of these words would be different if playing in a non-English speaking country like France for example. But we expect the scoreboard to look substantially the same.

For Updating the Scoreboard during a game that is not a tiebreaker, we only use natural counting numbers for sets and games, and use the numbers {0, 15, 30, 40} for scoring within a game. The only exception is that when one player has an advantage, their score will be `"AD"`, and the other player's score will be `"--"`. The count of the games and sets will be done with ordinary counting numbers 0, 1, 2, 3, ...

A method called `toString()` will be provided in your skeleton code for printing out the scoreboard in a very precise format. This will enable us to compare the correct output the output you generate. The player serving next will be indicated by preceding their name with the string `"S>"`.

A scoreboard could look like any of the following:

```
S> Alice          0  0      0
   Betty          0  0     30
```

This means Alice is serving and Betty has a score of 30, so the corresponding callout would be "Love-Thirty" (note that there is a convention to always announce the score of the person serving first). For the following scoreboard

```
   Alice          0  1      0
S> Betty          0  0      0
```

it must be the case that Alice just won a game, so the callout would be "Game: Alice". Note that it is not always possible to determine a callout by looking at the scoreboard, such as the following situation:

```
S> Alice          0  1      0
   Betty          0  1      0
```

There are two possibilities: Either Alice won before Betty or Betty won before Alice, and we don't know which one holds. However knowing the history of the points won enables us to make the correct callout, and the scoreboard is also determined by the history of the points won.

Specification

The specification for this assignment includes this document, the online Javadoc, and any "official" clarifications announced on Piazza.

Where's the main() method?

There isn't one! Like most Java classes, this isn't a complete program and you can't "run" it by itself. It's just a single class, that is, the definition for a type of object that might be part of a larger system. To try out your class, you can write a test class with a main method like the examples below in the getting started section.

There will also be an automated test suite running on Gradescope for this assignment, which will perform a lot of functional tests, but when you are developing and debugging your code at first you'll always want to have some simple test cases of your own, as in the getting started section below.

Suggestions for getting started

Smart developers don't try to write all the code and then try to find dozens of errors all at once; they work incrementally and test every new feature as it's written. Here is example of some incremental steps you could take in writing this class.

- Create a new, empty project and then add a package called hw2. Be sure to choose "Don't Create" at the dialog that asks whether you want to create `module-info.java`.
- Create a package named `hw2`. Download the skeleton code version of Tennis.java and copy it into your project. The easiest way to do this is by drag-and-dropping the file into src/hw2 in Eclipse.
- At this point, the `Tennis` class will not even compile. Getting the code to compile should be your top priority. While reading the javadoc, add stubs for all the required methods and the constructor. Document each one. For methods that need to return a value, just return a "dummy" value (e.g., 0 or false) as a placeholder for now. Continue until there are no compile errors in the project. At this point your code should pass all the tests that begin with `testL0`.
- Try a very simple test like this:

```
public class SimpleTests {
    public static void main(String[] args) {
        Tennis game = new Tennis("Alice", "Betty", false, false, false);
        System.out.println(game);
    }
}
```

You should see an output string similar to this, produced by the `toString()` method:

```
S> Alice         0  0      0
   Betty         0  0      0
```

Note that in printing the variable `game`, the println method automatically invokes the method `game.toString()`. The toString method just calls your other accessor methods to get the values to put in the string.

- Think about the constructor specified in the javadoc. They need to "memorize" the names of the players (we recommend Alice and Betty or Allan and Bobby). The constructors also need to specify three parameters which determine what kind of tennis match is being played.

  - `isBestOfFive` is a boolean which, if true, means we are playing a best-of-five sets match, and if false, means we are playing a best of three sets match. This has implications for how many sets one needs to win in order to win the match.
  - `isPlayingTiebreaks` is a boolean which determines whether we are using the Advantage system in which one must win a set by two games, or the Tiebreak system in which we play a tiebreak game if the score gets to 6-6.
  - `isGrandSlam` is a boolean which only has an effect when isPlayingTiebreaks is true, and in that case it determines whether the final game of the final set needs 7 points to win, or at least 10. If true, we need 10 sets to win the final set of a grand slam game. Otherwise we need only 7.

  These booleans should be stored in your class. Make sure playerA always serves first in any match (this is a rather arbitrary choice which holds for this assignment but not in real life).

- Next you could think about counting the sets, games and points. You'll need three instance variables for these three values, per player, and their values (or a string that depends on their value) should be returned by the corresponding accessor methods:

  - `public int getPlayerASets()`
  - `public int getPlayerAGames()`
  - `public int getPlayerAScore()`

and a similar set of methods for playerB. Remember that accessor methods never modify instance variables, they only observe them and return information. You'll also need a method `getPlayerAServing()` which returns a boolean which is true when playerA is serving next.

- The mutator methods for this project are

  - `public void winPoint(boolean playerAWins)`
  - `public void winGame(boolean playerAWins)`
  - `public void winSet(boolean playerAWins)`

  These cause the scores to change according to the rules discussed above.

- Familiarize yourself with the naming system for tests. The test names begin with the prefix `testLN` where N is 0, 1, 2, 3 or 4. This is followed by three letters that are either T or F, and the first one indicates true or false for `isBestOfFive`, the second one `isPlayingTiebreaks` and the third one `isGrandSlam`. Finally there is a number (optionally followed by a letter) for distinguishing tests.

  - `testL0` tests have to do with checking that you got the structure of the class correct, including the public method names and access modifiers, and just ensuring that your code compiles.
  - `testL1` tests check that your code can count sets correctly and determine the winner of the match based on tracking sets alone.
  - `testL2` tests have to do with checking that your code can count games correctly and use them to decide correctly when a set has been won, under the Advantage system. If a set has been won one needs to also determine whether the match has been won. Your code should not need to handle any tiebreaking rules in order to pass the testL2 tests.

  - `testL3` tests have to do with checking that your code can count points correctly within a non-tiebreaking game, and use them to decide correctly when a game has been won, and if so, whether a set has been won, and if so, whether the match has been won.
  - `testL4` tests introduce the concept of tiebreaking, and require your winGame and winSet methods to become aware of what tiebreaking means, and when to use tiebreaking. It also introduces the concept of a Grand Slam. This just allows your code to be aware of whether a grand slam match is being played and if so, correctly handle the final game in the final set according to the rules for grand slam tiebreaking.

- Writing your own tests and using TDD is a great idea, however we are not requiring you to submit a file containing your tests. Three methods have been provided in the skeleton file to make testing easier for you.

  - `runSets("a");`
  - `runGames("abb");`
  - `runPoints("abab");`

  The argument to each of these is a string containing lowercase a's and b's, each of which indicates which player (playerA or playerB) won the corresponding unit of play. For example, the call to runSets with an argument of "a" means playerA won a set. The call to runGames with an argument of "abb" means that playerA won the first game, and the next two games were won by playerB. Finally, the call to runPoints with the argument "abab" means that playerA won a point, then playerB won the next point, then playerA won the third point, and playerB the fourth. It is easy to set up a scenario in a tennis match by calling runSets with a suitable argument, then calling runGames to reflect the situation in a set, and then calling runPoints to create a scoreline. Then you can call `getCallOut()` and `toString()` to see whether they print what you expect the score to be.

- First implement the winSet method, and then test your code, and then run your code on the autograder to verify that you're passing all the testL1 tests. The winSet method should adjust the values of the scores. It should also determine whether the match has been won, and this condition will depend on whether it is a best-of-three sets match or a best-of-five sets match. One more thing to look out for is to make

sure the generated callout says "Game and Set: *playerName*" if the match has not be won, but says "Game, Set and Match: *playerName*" when the match has been won.

- Next, implement the winGame method (assuming that tiebreaks do not exist), and then run your code on the autograder to verify that you're passing all the testL1 and testL2 tests. The winGame method should adjust the values of the scores, and determine whether the set has been won. Pay attention to the rules for winning a set when we are playing Advantage Sets (but not Tiebreaker Sets). The winGame method should also change the indication of who is serving next. Finally it should generate a callout which says the game has been won, like "Game: *playerName*", but it should only do so if winning the game in this particular case does not mean the set has been won. This is because if the set has been won, winSet will generate a more informative message.

- Next, implement the winPoint method, and then run your code on the autograder to verify that you're passing all the testL1, testL2 and testL3 tests. Obviously the winPoint method should change the scores, and then check whether the game has been won. It should also generate callouts, stating the score of the server first, unless the game is a tiebreaker, in which case a different callout system is used.

- Fourth, we want to introduce the concept of tiebreaks. We need to include code in `winGame` that detects when a tiebreak should be played (whether the next game should be a tiebreaker). A tiebreaker should be played when the score is 6-6 and the boolean `isPlayingTiebreaks` is true. Then while playing a tiebreak game, one must win by attaining 7 points with a leading margin of 2, unless it is the final game of a grand slam, in which case one must win by attaining 10 points with a leading margin of 2.

  When a tiebreak game is being played, a few things work differently, such has how the scores are called (using normal numbers instead of nonconsecutive numbers), and also the fact that the serving player changes every two points, with the exception that the starting server serves only one point. That is, if player B serves first, then the service order will be:

  B A A B B A A B B A A ... and so on.

  And if A serves first, then the service order will be

A B B A A B B ... and so on.

Also the conditions for winning a tiebreak game are different from those of a normal game.

Please note that the alternation of the person serving during a tiebreaker game does not in any way affect the person who starts serving the game after the tiebreaker. If one player serves first in the tiebreaker, the other player should serve first in the game following the tiebreaker, regardless of what happens during the tiebreaker.

Also for the purposes of this assignment, assume that the person serving is never changed by calling `winSet`. That is, if we assume someone won a set by directly calling `winSet`, we are effectively assuming that they won the set with an even number of games played in that set, so that the set does not change who is serving.

- In all cases when a match has been won, the class should "lock up" and refuse to make any further changes to the scores. After the match is won, the three methods winPoint, winGame and winSet should not have any effect.

Requirements

The requirements for the class are embodied in the public API, so it is not mandatory that you use the ideas above. But they could be very helpful! And, of course, if you end up with a lot of unnecessarily duplicated code the grader may take off some points for style.

The autograder will be available on Gradescope. There are many test cases so there may be an overwhelming number of error messages. Always start reading the errors at the top and make incremental corrections in the code to fix them.

More about grading

This is a "regular" assignment so we are going to read your code. Your score will be based partly (about two thirds) on the autograder's functional tests and partly on the TA's assessment of the quality of your code. This means you can get partial credit even if you have errors, and it also means that even if you pass all the autograder tests you can still lose points. Are you doing things in a simple and direct way that makes sense? Are you defining redundant instance variables?

General principles to observe:

- Use instance variables only for the "permanent" state of the object, use local variables for temporary calculations within methods.
- You will lose points for having unnecessary instance variables
- All instance variables should be private.
- Accessor methods should not modify instance variables.

See the "Style and documentation" section below for additional guidelines.

We reserve the right to add more tests by the autograder other than the ones you see before the deadline.

Restrictions on Java Features For this Assignment

There are no restrictions on the Java features you can use to implement this assignment, except that you may not write any loops, and you may not write more than the one class specified in the javadoc. Also, see above the above section about using good programming practices. You may not use any outside libraries (i.e., external libraries such as Apache Commons; otherwise, we won't be able to compile your code when grading.

Style and documentation

Roughly 15% of the points will be for documentation and code style. Here are some general requirements and guidelines:

- Each class, method, constructor and instance variable, whether public or private,

must have a meaningful javadoc comment. The javadoc for the class itself can be very brief, but must include the `@author` tag. The javadoc for methods must include @param and `@return` tags as appropriate.

- Try to briefly state what each method does in your own words. There is no rule against copying the descriptions from the online documentation. However: do not literally copy and paste from this pdf! This leads to all kinds of weird bugs due to the potential for sophisticated document formats like Word and pdf to contain invisible characters.

- Run the javadoc tool and see what your documentation looks like! (You do not have to turn in the generated html, but at least it provides some satisfaction :) All variable names must be meaningful (i.e., named for the value they store). Your code should not be producing console output. You may add println statements when debugging, but you need to remove them before submitting the code. Internal (//-style) comments are normally used inside of method bodies to explain how something works, while the Javadoc comments explain what a method does. (A good rule of thumb is: if you had to think for a few minutes to figure out how something works, you should probably include an internal comment explaining how it works.)

- Internal comments always precede the code they describe and are indented to the

same level.

- Use a consistent style for indentation and formatting.

- Note that you can set up Eclipse with the formatting style you prefer and then use Ctrl-Shift-F to format your code. To play with the formatting preferences, go to Window>Preferences->Java->Code Style->Formatter and click the New button to create your own "profile" for formatting.

If you have questions

For questions, please see the Piazza Q&A pages and click on the folder hw2. If you don't find your question answered, then create a new post with your question. Try to state the question or topic clearly in the title of your post, and attach the tag hw2. But remember, do not post any source code for the classes that are to be turned in. It is fine to post source code for general Java examples that are not being turned in. (In the Piazza editor, use the button labeled "pre" to have Java code formatted the way you typed it.)

If you have a question that absolutely cannot be asked without showing part of your source code, make the post "private" so that only the instructors and TAs can see it. Be sure you have stated a specific question; vague requests of the form "read all my code and tell me what's wrong with it" will generally be ignored. Of course, the instructors and TAs are always available to help you. See the Office Hours section of the syllabus to find a time that is convenient for you. We do our best to answer every question carefully,

short of actually writing your code for you, but it would be unfair for the staff to fully review your assignment in detail before it is turned in.

Any announcements from the instructors on Piazza are considered to be part of the official spec, and you may lose points if you ignore them. Such posts will always be sent as an announcement on Piazza. (We promise that no official clarifications will be posted within 24 hours of the early due date.)

What to turn in

Note: You will need to complete the "Academic Dishonesty policy questionnaire," found on the Assignments page on Canvas, before you are given credit for this assignment.

Please submit, on Gradescope, just the Tennis.java file.